

CSV and TSV Files



Tabular data formats



CSV (Comma-Separated Values) and TSV (Tab-Separated Values) are plain text formats for storing tabular data, differing mainly in their delimiter: CSV uses commas, while TSV uses tabs to separate fields. TSV is often simpler to parse because tabs appear less frequently in text than commas, avoiding complex escaping, making TSV better for large datasets or machine processing, whereas CSV's comma-escaping mechanism makes it better for human-edited documents like spreadsheets

.tsv



A **.tsv file (Tab-Separated Values)** is a plain text file used for storing tabular data, like a spreadsheet, where each column's data is separated by a single tab character, and each row is a new line, making it great for data exchange between different programs. You can open and edit TSV files with simple text editors (Notepad, TextEdit) or import them into spreadsheet programs like Microsoft Excel, Google Sheets, and LibreOffice Calc, which automatically recognize the tab as a delimiter.

Key Characteristics:

Format: Plain Text: It's human-readable and not proprietary to a single application.

File Extension: .tsv

Purpose: Data storage, transfer, and analysis across various software.

Compatibility: Works with virtually any program that handles data, from simple text editors (Notepad) to complex databases.

Tab Delimited: Uses tabs to separate fields, which avoids issues with commas appearing within data, unlike CSV files.

Structured Data: Organizes data into rows (records) and columns (fields).

.tsv



Name	Age	City
John Doe	30	New York
Jane Smith	25	Los Angeles

This example uses arrows (→) to represent invisible tab characters for clarity.



Sepal length→Sepal width→Petal length→Petal width→Species

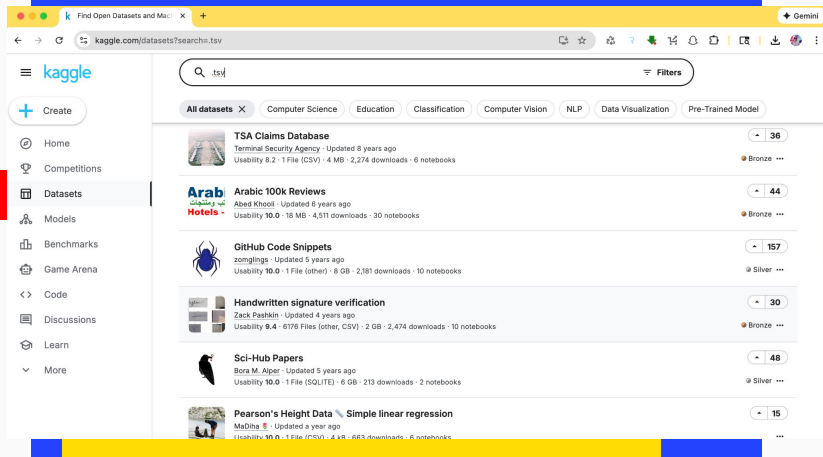
5.1→3.5→1.4→0.2→I. setosa

4.9→3.0→1.4→0.2→I. setosa

4.7→3.2→1.3→0.2→I. setosa

.tsv and machine learning

<https://opensource.ebay.com/tsv-utils/docs/comparing-tsv-and-csv.html>



.tsv files are frequently used in data science and machine learning for several reasons:

Data Exchange: They provide a simple, universal format for transferring data between different programming languages, databases, and spreadsheet programs without compatibility issues.

Cleaner Parsing: By using tabs as delimiters, they reduce ambiguity when the actual data contains commas (which is common in textual data for natural language processing tasks like sentiment analysis).

Human-Readability: The plain text structure is easy to inspect and edit manually in any text editor.

Efficiency: They are lightweight and can be processed efficiently, which is suitable for large datasets often encountered in machine learning environments.

Let's practice with python pandas to work with data in tsv files:



The screenshot displays a JupyterLab workspace with three tabs: `loadData.py`, `querying.py`, and `titanic.tsv`. The `loadData.py` tab contains the following code:

```
1 import pandas as pd
2
3 # Load the TSV file WITHOUT headers
4 df = pd.read_csv("titanic.tsv", sep="\t", header=0)
5
6 # Display basic information about the dataset
7 print("Shape (rows, columns):", df.shape)
8 print("\nFirst 5 rows:")
9 #by default head() shows first 5 rows
10 print(df.head())
11 print("\nFirst 10 rows:")
12 print(df.head(10))
```

The `querying.py` tab contains the following code:

```
1 #we're creating a pandas dataframe: A pandas D
2 #in the pandas library for Python used for dat
3 #It is a two-dimensional, size-mutable, and po
4 # with labeled axes (rows and columns), simila
5 import pandas as pd
6
7 columns = [
8     "PassengerId",
9     "Survived",
10    "Pclass",
11    "Name",
12    "Sex",
```

The `titanic.tsv` tab shows the first 12 rows of the dataset:

PassengerId	Survived	Pclass	Name	Sex
1	0	3	Braund, Mr. Owen Harris	male
2	1	1	Cumings, Mrs. John Bradley	female
3	1	3	Heikkinen, Miss. Laina	female
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female
5	0	3	Allen, Mr. William Henry	male
6	0	3	Moran, Mr. James	male
7	0	1	McCarthy, Mr. Timothy J	male
8	0	3	Palsson, Master. Gosta	male
9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female
10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female
11	1	3	Sandstrom, Miss. Margaretha Johansson	female
12	1	1	Bonnell, Miss. Elizabeth	female

The terminal output at the bottom shows the execution of the code, displaying the shape of the dataframe and the first 10 rows:

```
[342 rows x 12 columns]
PassengerId  Survived  Pclass  Name  ...  Ticket  Fare  Cabin  Embarked
1            2         1       1  Cumings, Mrs. John Bradley (Florence Briggs Th...  ...  PC 17599  71.2833  C85        C
```

.CSV



A **CSV (Comma-Separated Values)** file is a simple, plain text file format for storing tabular data, where each row is a data record and values within that row are separated by commas, making it easy to exchange data between different applications like spreadsheets (Excel, Google Sheets) and databases. It's a lightweight, universally compatible format, ideal for importing/exporting large datasets, but it doesn't support complex formatting like colors, formulas, or multiple sheets, only raw data.

Key Characteristics:

Format: Plain text, using commas as delimiters for columns and newlines for rows.

File Extension: .csv.

Purpose: Data storage, transfer, and analysis across various software.

Compatibility: Works with virtually any program that handles data, from simple text editors (Notepad) to complex databases.

Limitations: Lacks advanced formatting (bold, colors, formulas) and only supports single sheets.

Common Uses:

Data Exchange: Moving data between different systems (e.g., importing customer lists).

Import/Export: Uploading or downloading product/order data for e-commerce platforms like Shopify.

Data Analysis: Importing raw data into Excel or Google Sheets for analysis.

.CSV



CSV

Name,Email,PhoneNumber

John Doe,john.doe@example.com,123-555-1234

Jane Smith,jane.smith@example.com,123-555-5678

Peter Jones,peter.jones@example.com,123-555-9012

When a field's data contains a comma or a space, the entire field is typically enclosed in double quotes to prevent the comma from being misinterpreted as a field separator. If a field itself contains a double quote, that quote is escaped by using two double quotes ("").

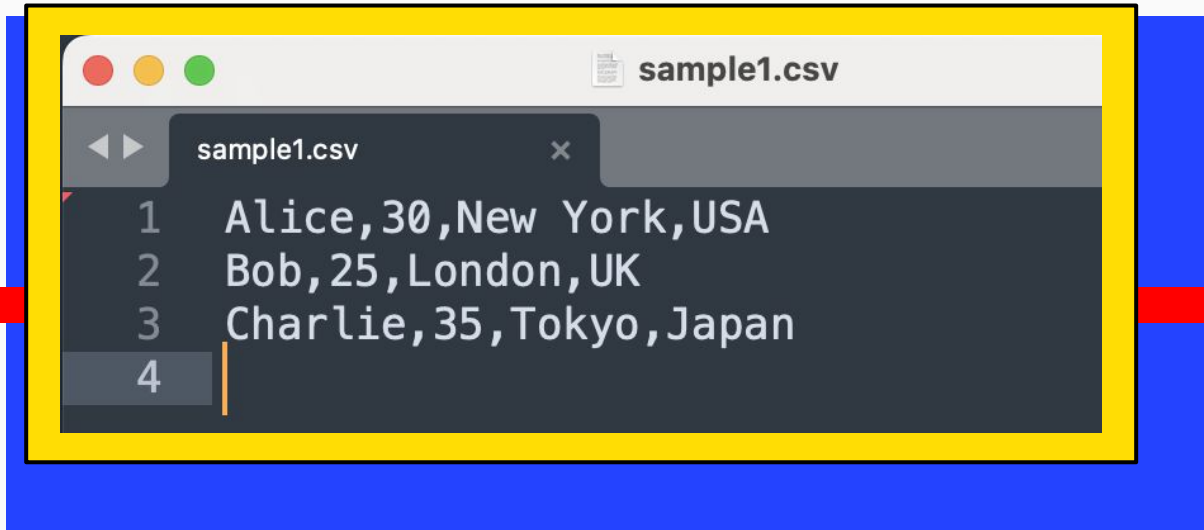
CSV

"Full Name","Address","Notes"

"John Doe","123 Main St, Anytown, USA","Likes the outdoors"

"Jane Smith","456 Oak Ave, Sometown, USA","Often says ""hello"" first"

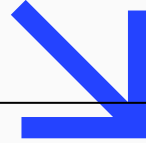
Let's try making our own csv file ↘



A screenshot of a text editor window titled "sample1.csv". The editor shows a CSV file with three rows of data. The first row is "1 Alice,30,New York,USA", the second row is "2 Bob,25,London,UK", and the third row is "3 Charlie,35,Tokyo,Japan". The fourth row is empty, with the number "4" in the first column. The editor has a dark background and a yellow border. A red horizontal bar is visible on the right side of the editor window.

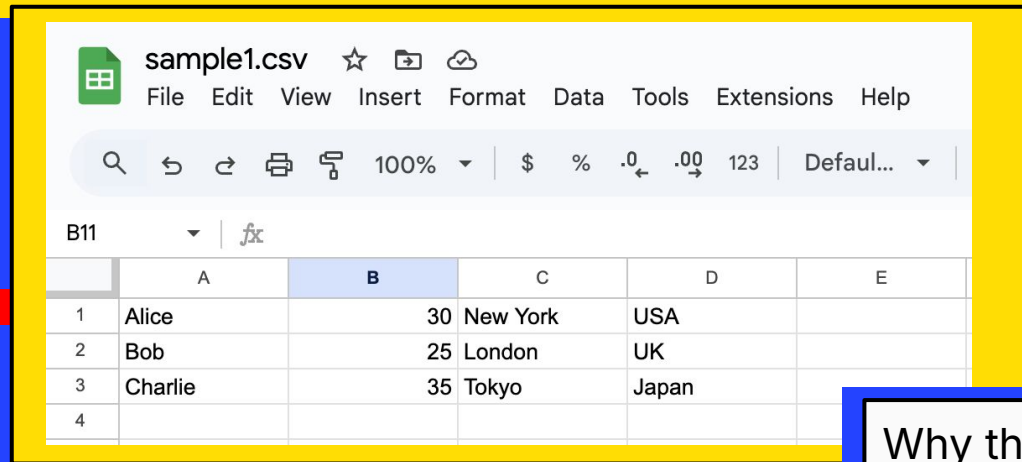
```
sample1.csv
1 Alice,30,New York,USA
2 Bob,25,London,UK
3 Charlie,35,Tokyo,Japan
4
```

What happens when we open our .csv file in a program like Excel or Google Sheets?



	A	B	C	D	E
1	Alice	30	New York	USA	
2	Bob	25	London	UK	
3	Charlie	35	Tokyo	Japan	
4					

We have rows and columns where our delimiters were



sample1.csv

File Edit View Insert Format Data Tools Extensions Help

100% \$ % .0 .00 123 Defaul...

B11

	A	B	C	D	E
1	Alice	30	New York	USA	
2	Bob	25	London	UK	
3	Charlie	35	Tokyo	Japan	
4					

Why this implied structure? How is it useful/important? What's missing?

(Is it the only way)?

Let's add headers to our csv file to contextualize the data

```
sample1.csv
1 // Header row (column names)
2 Name, Age, City, Country
3
4 // Data rows (records)
5 Alice, 30, New York, USA
6 Bob, 25, London, UK
7 Charlie, 35, Tokyo, Japan
```

sample1

File Edit View Insert Format Data Tools Extensions Help

100%

D13

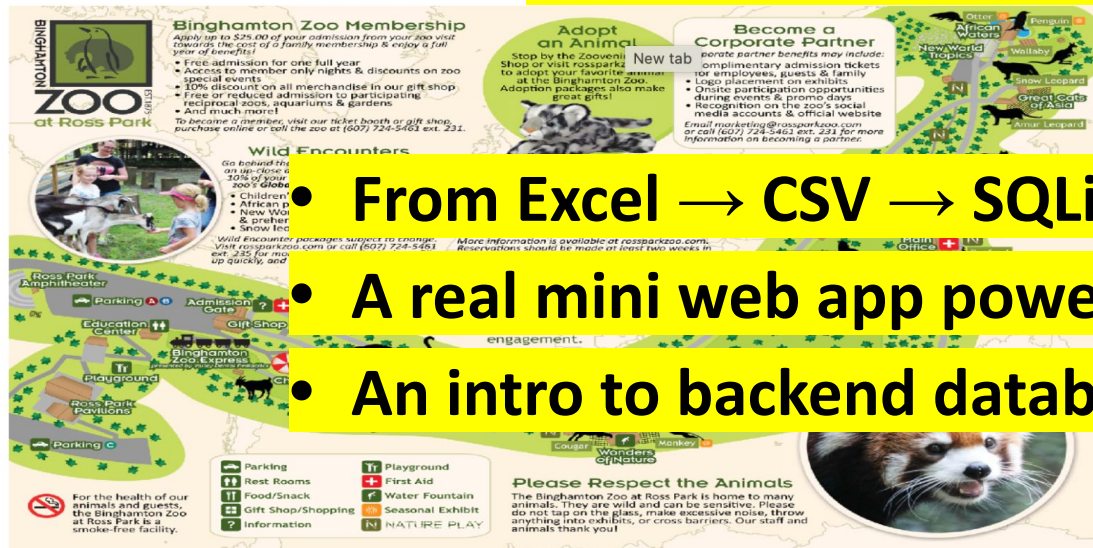
	A	B	C	D	E
1	// Header row (column names)				
2	Name	Age	City	Country	
3					
4	// Data rows (records)				
5	Alice	30	New York	USA	
6	Bob	25	London	UK	
7	Charlie	35	Tokyo	Japan	



Welcome to

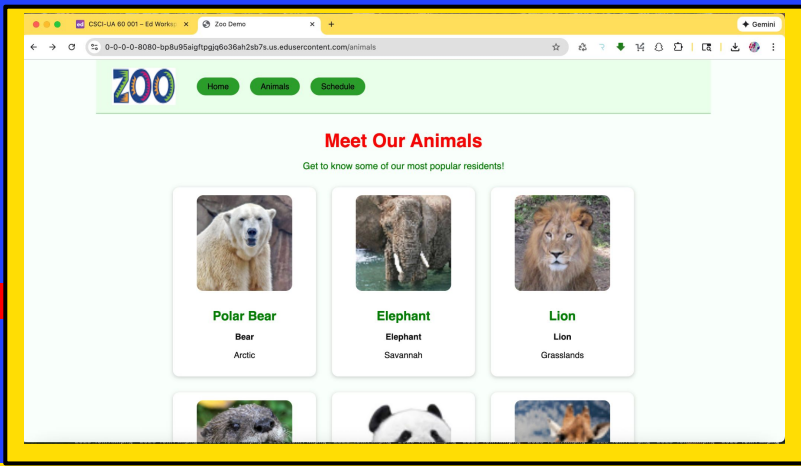
Explore the oldest zoo in

Python + Flask + SQLite Zoo Demo Edition



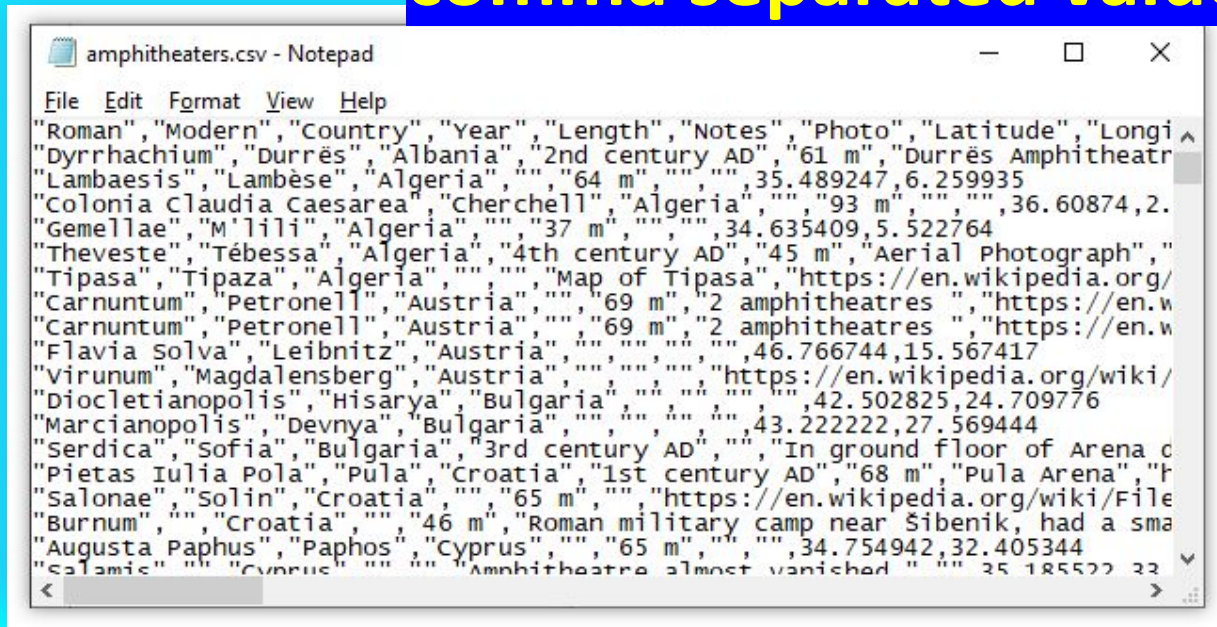
- From Excel → CSV → SQLite → Flask
- A real mini web app powered by animal data
- An intro to backend database systems

<https://edstem.org/us/courses/91931/workspaces/pvZj1XsdSCn4GbUhlqaFyjHqp0wc9UoU>



Let's start With CSV files!

comma separated value



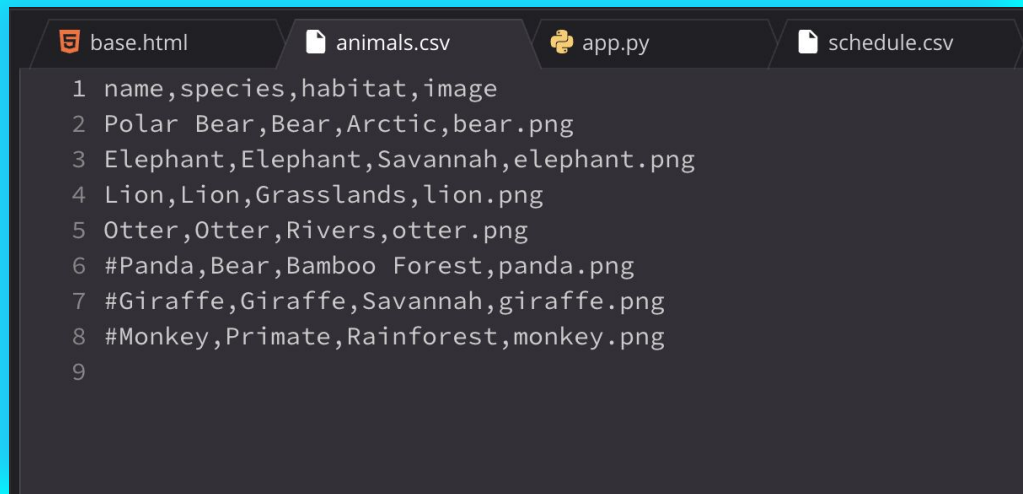
```
amphitheaters.csv - Notepad
File Edit Format View Help
"Roman","Modern","Country","Year","Length","Notes","Photo","Latitude","Longitude"
"Dyrrhachium","Durrës","Albania","2nd century AD","61 m","Durrës Amphitheatre","https://en.wikipedia.org/wiki/File:Durrës_Amphitheatre.jpg",35.489247,6.259935
"Lambaesis","Lambèse","Algeria","","64 m","","35.489247,6.259935
"Colonia Claudia Caesarea","Cherchell","Algeria","","93 m","","36.60874,2.635409
"Gemellae","M'lili","Algeria","","37 m","","34.635409,5.522764
"Theveste","Tébessa","Algeria","4th century AD","45 m","Aerial Photograph","https://en.wikipedia.org/wiki/File:Theveste_Aerial.jpg",36.60874,2.635409
"Tipasa","Tipaza","Algeria","","","Map of Tipasa","https://en.wikipedia.org/wiki/File:Tipaza.jpg",36.60874,2.635409
"Carnuntum","Petronell","Austria","","69 m","2 amphitheatres","https://en.wikipedia.org/wiki/File:Carnuntum.jpg",48.333333,16.666667
"Flavia solva","Leibnitz","Austria","","","","46.766744,15.567417
"Virunum","Magdalensberg","Austria","","","","https://en.wikipedia.org/wiki/File:Magdalensberg.jpg",46.766744,15.567417
"Diocletianopolis","Hisarya","Bulgaria","","","","42.502825,24.709776
"Marcianopolis","Devnya","Bulgaria","","","","43.222222,27.569444
"Serdica","Sofia","Bulgaria","3rd century AD","","In ground floor of Arena complex","https://en.wikipedia.org/wiki/File:Serdica.jpg",42.502825,24.709776
"Pietas Iulia Pola","Pula","Croatia","1st century AD","68 m","Pula Arena","https://en.wikipedia.org/wiki/File:Pula_Arena.jpg",45.50505,13.70505
"Salonae","Solina","Croatia","","65 m","","https://en.wikipedia.org/wiki/File:Salonae.jpg",45.50505,13.70505
"Burnum","","Croatia","","46 m","Roman military camp near Šibenik, had a small amphitheatre","https://en.wikipedia.org/wiki/File:Burnum.jpg",45.50505,13.70505
"Augusta Paphus","Paphos","Cyprus","","65 m","","34.754942,32.405344
"Salamis","","Cyprus","","","Amphitheatre almost vanished","https://en.wikipedia.org/wiki/File:Salamis.jpg",35.185522,33.425556
```

to CSV

<https://medium.com/@TheWuraolah/how-to-extract-data-from-an-api-and-save-it-as-a-csv-910231b53193>

CSV → Python

- CSV keeps the rows/columns model from Excel
- Python's `csv.DictReader` reads each row as a python dictionary object



```
base.html animals.csv app.py schedule.csv
1 name,species,habitat,image
2 Polar Bear,Bear,Arctic,bear.png
3 Elephant,Elephant,Savannah,elephant.png
4 Lion,Lion,Grasslands,lion.png
5 Otter,Otter,Rivers,otter.png
6 #Panda,Bear,Bamboo Forest,panda.png
7 #Giraffe,Giraffe,Savannah,giraffe.png
8 #Monkey,Primate,Rainforest,monkey.png
9
```

Loading CSV in Python

```
import csv

def load_csv(filepath):
    data = []

    # Always open CSVs with newline='' to avoid extra blank lines on Windows
    with open(filepath, newline='', encoding="utf-8") as f:
        reader = csv.DictReader(f)

        # Each row becomes a dictionary:
        # { "column_name": "value", ... }
        for row in reader:
            data.append(row)

    return data
```

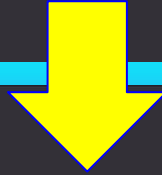


- animals.csv stores zoo data

- Python loads it into a list of dictionaries

- These dictionaries feed the Flask template

```
def load_animals():
    animals = []
    with open("animals.csv", newline='', encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            animals.append(row)
    return animals
```



```
1 {% extends "base.html" %}
2
3 {% block content %}
4 <section id="topSection">
5   <h1>Meet Our Animals</h1>
6   <p>Get to know some of our most popular residents!</p>
7 </section>
8
9 <div class="animal-grid">
10   {% for a in animals %}
11     <div class="animal-card">
12       
13       <h2>{{ a.name }}</h2>
14       <p><strong>{{ a.species }}</strong></p>
15       <p>{{ a.habitat }}</p>
16     </div>
17   {% endfor %}
18 </div>
19 {% endblock %}
20
21
```

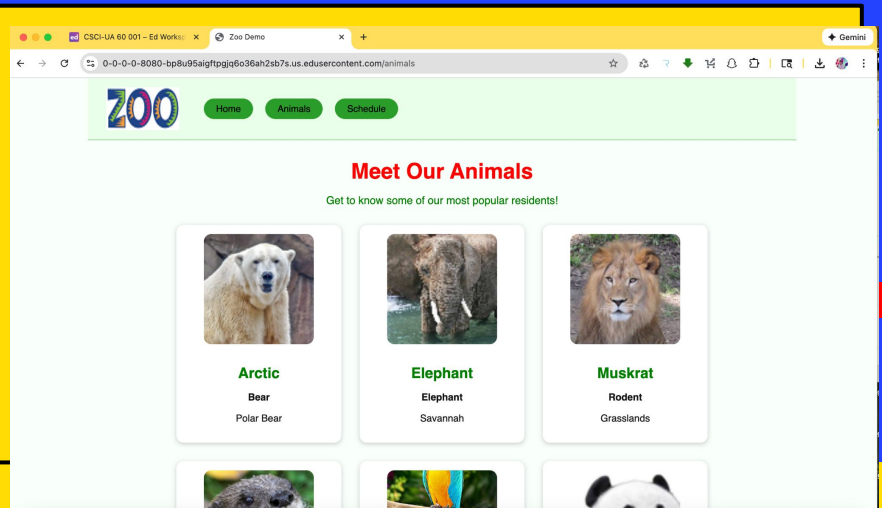
What happens when we change the order of the data in the .csv file? What does that tell us about the structure ?

animals.csv

```
1 name,species,habitat,image
2 Arctic,Bear,Polar Bear,bear.png
3 Elephant,Elephant,Savannah,elephant.png
4 Muskrat,Rodent,Grasslands,lion.png
5 Otter,Otter,Rivers,otter.png
6 Parrot,Bird,Rainforest,parrot.jpg
7 Panda,Bear,Bamboo Forest,panda.png
8 Giraffe,Giraffe,Savannah,giraffe.png
9 Monkey,Elephant,Rainforest,monkey.png
10
11
```

user@sahara:~

```
127.0.0.1 - - [19/Jan/2026 11:51:47] "GET /animals HTTP/1.1" 200 -
127.0.0.1 - - [19/Jan/2026 11:51:47] "GET /static/images/logo.png HTTP/1.1" 200 -
127.0.0.1 - - [19/Jan/2026 11:51:47] "GET /static/images/bear.png HTTP/1.1" 200 -
127.0.0.1 - - [19/Jan/2026 11:51:47] "GET /static/main.css HTTP/1.1" 200 -
127.0.0.1 - - [19/Jan/2026 11:51:47] "GET /static/images/elephant.png HTTP/1.1" 200 -
127.0.0.1 - - [19/Jan/2026 11:51:47] "GET /static/images/panda.png HTTP/1.1" 200 -
127.0.0.1 - - [19/Jan/2026 11:51:47] "GET /static/images/monkey.png HTTP/1.1" 200 -
127.0.0.1 - - [19/Jan/2026 11:51:47] "GET /static/images/parrot.jpg HTTP/1.1" 200 -
127.0.0.1 - - [19/Jan/2026 11:51:47] "GET /static/images/giraffe.png HTTP/1.1" 200 -
127.0.0.1 - - [19/Jan/2026 11:51:47] "GET /static/images/lion.png HTTP/1.1" 200 -
127.0.0.1 - - [19/Jan/2026 11:51:47] "GET /static/images/otter.png HTTP/1.1" 200 -
^C[user@sahara:~]$
```



Let's practice with python load_csv() to work with data in .csv files:



The screenshot shows a web-based code editor interface. The browser address bar shows the URL: `edstem.org/us/courses/91931/workspaces/pad2hjKGd6ol3trffGogLdU7lgmtaTp8`. The editor has a sidebar on the left with a file explorer showing `animals.csv` and `app.py`. The main editor area is split into two panes. The left pane shows the contents of `animals.csv`:

```
1 name,species,habitat,image
2 Polar Bear,Bear,Arctic,bear.png
3 Elephant,Elephant,Savannah,elephant.png
4 Lion,Lion,Grasslands,lion.png
5 Otter,Otter,Rivers,otter.png
6 Panda,Bear,Bamboo Forest,panda.png
7 Giraffe,Giraffe,Savannah,giraffe.png
8 Monkey,Primate,Rainforest,monkey.png
9 Parrot,Bird,Rainforest,parrot.jpg
10
```

The right pane shows the contents of `app.py`:

```
26 for animal in animals:
27     print(f'{animal["name"]} lives in {animal["habitat"]}')
28
29 #slicing
30 for animal in animals[:3]:
31     print(animal["name"], animal["habitat"])
32 #Animals 2 through 4
33 for animal in animals[1:4]:
34     print(animal["name"])
35 #Slice the last items (very useful pattern)
36 for animal in animals[-2:]:
37     print(animal["name"])
```

Below the code panes is a terminal window showing the command `python3 app.py` and its output:

```
[user@sahara ~]$ python3 app.py
[{'name': 'Polar Bear', 'species': 'Bear', 'habitat': 'Arctic', 'image': 'bear.png'}, {'name': 'Elephant', 'species': 'Elephant', 'habitat': 'Savannah', 'image': 'elephant.png'}, {'name': 'Lion', 'species': 'Lion', 'habitat': 'Grasslands', 'image': 'lion.png'}, {'name': 'Otter', 'species': 'Otter', 'habitat': 'Rivers', 'image': 'otter.png'}, {'name': 'Panda', 'species': 'Bear', 'habitat': 'Bamboo Forest', 'image': 'panda.png'}, {'name': 'Giraffe', 'species': 'Giraffe', 'habitat': 'Savannah', 'image': 'giraffe.png'}, {'name': 'Monkey', 'species': 'Primate', 'habitat': 'Rainforest', 'image': 'monkey.png'}, {'name': 'Parrot', 'species': 'Bird', 'habitat': 'Rainforest', 'image': 'parrot.jpg'}]
Polar Bear
Elephant
Lion
Otter
Panda
```

The terminal window also shows the prompt `user@sahara:~` and the command `python3 app.py`.